

Recherche d'Information (RI)

Les Modèles de RI

*Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'informatique
à l'UFR-SEA / UJKZ*

Mai 2022

<http://kisitokab.xyz>

- Introduction à la RI:
 - notions fondamentales et les concepts
 - problématique
 - historique
- Représentation de documents et Indexation
- **Modèles de RI**
- Traitement des requêtes et Évaluation de la RI
- Moteurs de recherche sur le Web

Objectif

- À la fin de cette unité, vous devriez être capable de:
 - Maîtriser le fonctionnement du modèle Booléen.
 - Maîtriser le fonctionnement du modèle Vectoriel.
 - Maîtriser le fonctionnement du modèle probabiliste.

Chapitre 3 : Les Modèles de RI

Introduction

- Nous présentons les modèles de recherche usuels tels que
 - les modèles *booléens*,
 - *vectoriels* et
 - *probabilistes*.

Chapitre 3 : Les Modèles de RI

- Introduction
- Nous établirons aussi, les conditions que les fonctions *vectorielles* et *probabilistes*, estimant la similarité entre une requête et un document, doivent respecter.
- Nous présenterons la **base formelle** pour le développement de nouveaux modèles de recherche.

3.1) Modèles de recherche

3.1) Modèles de recherche

- 3.1 Modèles de recherche

- Les modèles de recherche sont des programmes qui aident les utilisateurs à trouver les informations qu'ils recherchent dans une collection de documents textuels ou éventuellement multimédias.
- Pour une demande d'information donnée, le but de ces modèles est de retourner un sous-ensemble de documents de la collection qui pourraient contenir l'information recherchée.

3.1) Modèles de recherche

- 3.1 Modèles de recherche

- Les **documents de ce sous-ensemble** qui contiennent effectivement l'information recherchée sont appelés ***documents pertinents***, les autres documents étant des ***documents non pertinents*** par rapport à la requête.
- Les différentes étapes de ce processus de recherche sont schématisées dans la figure 1.

3.1) Modèles de recherche



Fig. 1 – Les différentes étapes de la recherche d'information

3.1) *Modèles de recherche*

- La première phase est l'**indexation** et la **représentation** des documents, opérations explicitées au chapitre 2.
- La deuxième phase est la **formulation du besoin** d'information exprimé par les utilisateurs sous forme d'une requête simple.
- Enfin, la dernière étape consiste à **appairer** la requête aux documents prétraités et indexés.

3.1) Modèles de recherche

- Après cette phase, on peut aussi utiliser un processus de **dialogue interactif** avec l'utilisateur qui lui permet de mieux cerner son besoin d'information.
- Nous introduisons à présent les **trois familles** de modèles de recherche classiques utilisés en RI.
- (il existe d'autres types de modèles, comme les modèles fondés sur les **réseaux possibilistes** exposés dans (Brini *et al.* 2006) ou, plus récemment, sur la **théorie quantique** ; nous ne les abordons pas dans ce cours.).

3.1.1) Modèles booléens

3.1.1) Modèles booléens

- **3.1.1 Modèles booléens**

- Les modèles booléens de recherche sont fondés sur la logique booléenne et la théorie des ensembles.
- Pendant plusieurs dizaines d'années, et ce jusqu'aux années 1990, ces modèles ont constitué le cœur des systèmes de recherche d'information commerciaux.
- Ils ont été conçus de façon à répondre exactement aux requêtes formées par des expressions combinant des termes et des opérateurs logiques ET, OU et SAUF.
- Une façon efficace de traiter ce genre de requête est d'utiliser la structure d'index inversé présentée au chapitre 2.
- Dans ce cas, la recherche booléenne se réduit simplement à parcourir les listes de documents associées à chacun des termes de la requête, et de fusionner ensuite ces listes par rapport aux opérateurs logiques présents dans cette dernière.

3.1.1) Modèles booléens

- **3.1.1 Modèles booléens**
- **Par exemple**, pour une requête conjonctive formée de deux termes liés par l'opérateur ET, l'algorithme 2 montre une façon simple de fusionner les deux listes de documents associées aux termes de la requête.
- Cet algorithme initialise d'abord deux pointeurs $pt1$ et $pt2$ sur chacune des deux listes.
- À chaque itération, il compare ensuite les identifiants des documents référencés par les pointeurs ($pt1 \rightarrow IdDoc$ et $pt2 \rightarrow IdDoc$).
- Dans le cas d'égalité, l'identifiant trouvé est ajouté à la liste des documents répondant à la requête (Lr) et on avance ensuite les pointeurs d'un cran ;
- sinon c'est juste le pointeur pointant sur l'identifiant le plus petit qui est avancé.
- L'algorithme se termine lorsque l'un des deux pointeurs arrive à la fin de sa liste.

3.1.1) Modèles booléens

ALGORITHME 2. Algorithme de fusion de deux listes inversées de termes avec l'opérateur ET

Entrée : deux listes inversées LI_1 et LI_2

Initialisation : $L_r \leftarrow \emptyset$; $pt_1 \leftarrow Debut(LI_1)$; $pt_2 \leftarrow Debut(LI_2)$

tant que $(pt_1 \neq NULL) \wedge (pt_2 \neq NULL)$ **faire**

si $(pt_1 \rightarrow IdDoc) = (pt_2 \rightarrow IdDoc)$ **alors**

$L_r \leftarrow L_r \cup \{pt_1 \rightarrow IdDoc\};$

$pt_1 \leftarrow (pt_1 \rightarrow next);$

$pt_2 \leftarrow (pt_2 \rightarrow next);$

sinon si $(pt_1 \rightarrow IdDoc) < (pt_2 \rightarrow IdDoc)$ **alors**

$pt_1 \leftarrow (pt_1 \rightarrow next);$

sinon

$pt_2 \leftarrow (pt_2 \rightarrow next);$

fin

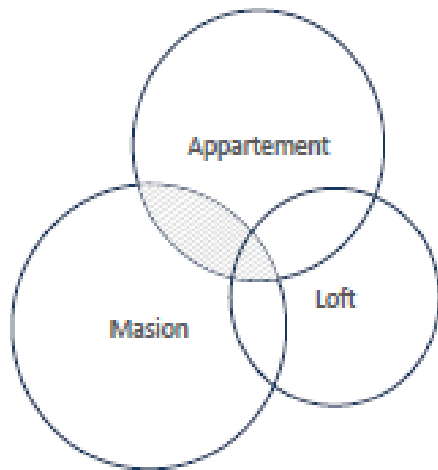
fin

Sortie : L_r

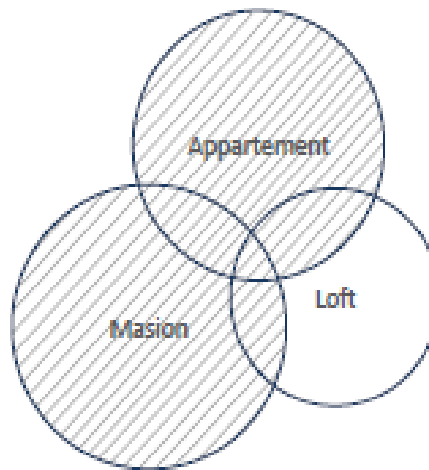
3.1.1) Modèles booléens

- **3.1.1 Modèles booléens**
- On voit bien ici que
- la recherche avec une requête conjonctive constitue une *intersection* entre les deux listes d'identifiants des termes de la requête ;
- les requêtes formées de deux termes et des opérateurs OU et SAUF vont correspondre respectivement à une *union* et une *exclusion* sur les listes de documents associées à ces termes.
- Les diagrammes de Venn de la figure 3.2 illustrent les résultats de recherche obtenus avec ces opérateurs.
- Dans cette figure, chaque disque représente l'ensemble des documents pertinents par rapport à un terme donné du vocabulaire, et les parties hachurées montrent les résultats de recherche obtenus avec les opérateurs concernés.

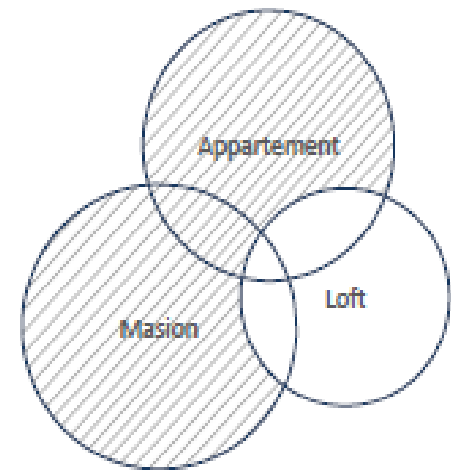
3.1.1) Modèles booléens



Maison ET Apparte-
ment



Maison OU Apparte-
ment



Maison OU Apparte-
ment SAUF Loft

3.1.1) Modèles booléens

- **3.1.1 Modèles booléens**

- Pour des requêtes **conjonctives**, la recherche est très rapide dans le cas où un des termes est rare, puisque dans ce cas on va parcourir la liste des documents du terme rare en premier et ne conserver que les identifiants de documents également présents dans l'autre liste.
- Cette propriété permet d'optimiser la recherche avec des requêtes **conjonctives** constituées de plus de deux termes, en traitant les listes de documents des termes ayant les plus petites fréquences documentaires, df, en premier.
- L'algorithme 2 peut aussi être facilement adapté au cas des requêtes formées d'une **disjonction** entre deux termes.
- Pour des requêtes plus complexes, on utilise souvent la distributivité entre les opérateurs ET et OU pour rendre ces dernières sous forme de conjonctions d'expressions plus simples d'un terme ou d'une disjonction de termes (l'expression booléenne constituant la requête est ainsi mise sous forme normale conjonctive)

3.1.1) Modèles booléens

- **3.1.1 Modèles booléens**
- Le modèle booléen peut être utilisé efficacement sur des collections de documents spécialisés dans le cas où les utilisateurs ont une bonne connaissance du vocabulaire associé à ces collections.

3.1.1) Modèles booléens

- **3.1.1 Modèles booléens**
- En contrepartie, l'inconvénient majeur de ce modèle est que, pour un besoin d'information particulier, il ne présente pas de liste ordonnée de documents, du plus pertinent au moins pertinent par rapport à la requête, rendant la recherche difficile pour des utilisateurs peu chevronnés.
- Ainsi, un document qui contient les termes de la requête mais porte, pour l'essentiel, sur une thématique différente de celle couverte par ces termes sera jugé aussi pertinent qu'un document dont la thématique principale est celle couverte par les termes de la requête.

3.1.1) Modèles booléens

- 3.1.1 Modèles booléens
- Le deuxième inconvénient est que le modèle booléen effectue des appariements exacts entre les termes des requêtes et les documents, ce qui ne permet pas le renvoi des documents pertinents ne contenant pas les termes exacts des requêtes booléennes saisies.
-

3.1.2) Modèles vectoriels

3.1.2) Modèles vectoriels

- 3.1.2 Modèles vectoriels

- Dans des très grandes collections de documents, le résultat de recherche pour une requête donnée dépasse généralement la capacité des utilisateurs à examiner tous les documents de l'ensemble retourné.
- Obtenir une liste de documents ordonnés (par similarité par rapport à la requête) est ainsi primordial pour trouver rapidement l'information recherchée et distinguer les documents selon qu'ils couvrent principalement ou partiellement le ou les thèmes (voire le ou les aspects) de la requête.

3.1.2) Modèles vectoriels

- 3.1.2 Modèles vectoriels

- Les modèles de recherche **vectoriels** et **probabilistes** résolvent ce problème en assignant simplement des **scores de similarité** à toutes les paires formées par une requête et un document d'une collection donnée.
- L'hypothèse de base des modèles vectoriels est que les requêtes peuvent être représentées dans le même espace vectoriel que les documents et que les documents de plus hauts scores sont ceux qui sont les plus susceptibles d'être pertinents par rapport à la requête.

3.1.2) Modèles vectoriels

- **3.1.2 Modèles vectoriels**

- Ainsi, pour chaque requête q on considère sa représentation vectorielle $\mathbf{q} \in \mathbb{R}^V$.
- Le but d'un modèle de recherche est alors de prédire pour la requête fixée un ordre sur les documents de la collection C en utilisant une fonction de score s .
- Formellement, cette fonction prend en entrée les deux représentations vectorielles, de la requête et d'un document de C , et renvoie un score réel reflétant la similarité entre ces deux vecteurs i.e. $s : \mathbb{R}^V \times \mathbb{R}^V \rightarrow \mathbb{R}$.

3.1.2) Modèles vectoriels

- 3.1.2 Modèles vectoriels

- Les premiers modèles vectoriels fixaient à l'avance l'expression analytique de la fonction de score à utiliser.
- Les moteurs de recherche vectoriels actuels ont de plus en plus recours aux techniques d'apprentissage automatique pour trouver une fonction de score à partir d'une combinaison de différentes fonctions de base.
- Ces systèmes trouvent automatiquement les paramètres associés aux fonctions intervenant dans la combinaison finale en utilisant une base d'entraînement dans laquelle on dispose d'un ensemble de requêtes ainsi que des documents pertinents et non pertinents par rapport à chacune de ces requêtes.

3.1.2) Modèles vectoriels

- Modèle vectoriel de Salton
- L'avantage de présenter les requêtes dans le même espace vectoriel que les documents est qu'on peut utiliser des mesures de similarité vectorielles simples pour estimer l'affinité numérique entre les vecteurs de même dimension (Salton 1968 ; Salton et McGill 1986).
- La mesure d'affinité la plus répandue en RI est la **similarité cosinus** qui détermine l'angle entre deux vecteurs (ceux d'un document et d'une requête en l'occurrence).

3.1.2) Modèles vectoriels

- **Modèle vectoriel de Salton**
- Dans ce cas, plus l'angle entre les vecteurs d'un document et de la requête est petit (correspondant à un score cosinus élevé) plus le document est supposé être pertinent par rapport à la requête.
- En particulier, le cosinus varie entre 0 et 1 lorsque l'on ne considère que des poids positifs ou nuls (ce qui est généralement réalisé en pratique).
- Le cosinus vaut alors 0 lorsque requête et document n'ont aucun mot en commun, et 1 lorsqu'ils contiennent exactement les mêmes mots, dans les mêmes proportions.

3.1.2) Modèles vectoriels

- Modèle vectoriel de Salton
- La recherche avec un modèle vectoriel, s'effectue donc en deux étapes :
- 1. Calculer les scores de similarité entre les documents d'une collection et une requête donnée.
- 2. Présenter à l'utilisateur le résultat de la recherche constitué des L premiers documents de plus haut score.
-

3.1.2) Modèles vectoriels

- Modèle vectoriel de Salton
-
- L'algorithme 3 présente l'implémentation d'un tel système dans le cas où la fonction de score est la mesure cosinus (équation 1).

3.1.2) Modèles vectoriels

ALGORITHME 3. Modèle vectoriel de recherche - implémentation du score cosinus (équation 3.1)

Entrée :

- une collection C de N documents et son index inversé associé ;
- un tableau contenant les facteurs de normalisation associés aux documents $(Norm_d[j])_{j \in \{1, \dots, N\}}$;
- $q = \{t_1^q, t_2^q, \dots, t_K^q\}$: une requête constituée de K termes.

Initialisation :

- $\forall j \in \{1, \dots, N\}, s[j] \leftarrow 0$; // le tableau qui contiendra les mesures de similarité
- $Norm_q \leftarrow 0$. // le facteur de normalisation associé à la requête

pour $i \in \{1, \dots, K\}$ **faire**

 Estimer $w_{t_i^q, q} \leftarrow p_{tf_{t_i^q, q}} \times p_{df_{t_i^q}}$; // (tableau 2.3, chapitre 2)

$Norm_q \leftarrow Norm_q + (w_{t_i^q, q})^2$;

pour tous les j **dans la liste des identifiants de** t_i^q **faire**

 Estimer $w_{t_i^q, d_j} \leftarrow n_{d_j} \times p_{tf_{t_i^q, d_j}} \times p_{df_{t_i^q}}$; // (tableau 2.3, chapitre 2)

$s[j] \leftarrow s[j] + w_{t_i^q, q} \times w_{t_i^q, d_j}$;

fin

fin

pour $j \in \{1, \dots, N\}$ **faire**

si $s[j] \neq 0$ **alors**

$s[j] \leftarrow s[j] / (\sqrt{Norm_q} \times \sqrt{Norm_d[j]})$;

fin

fin

Sortie : les L documents de plus haut score $s[.]$

3.1.2) Modèles vectoriels

- Modèle vectoriel de Salton
 - Comme on peut le constater, on a supposé ici que les documents de la collection ne sont pas modifiés au cours du temps et que l'index inversé des termes contient la fréquence documentaire ainsi que le nombre d'occurrences des termes dans les documents de la collection (notés respectivement par df et tf - voir unité 2).

3.1.2) Modèles vectoriels

- Modèle vectoriel de Salton
 - En utilisant ces fréquences et les nombres d'occurrences, on calcule, pour chaque terme t_i^q d'une requête q , les poids de ce terme dans la requête et les documents de la liste inversée de ce dernier (notés respectivement par $w_{t_i q}^q$ et $w_{t_i d_j}^q$).
 - Ces poids sont ensuite utilisés pour estimer des scores entre chaque document de la collection et la requête.

3.1.2) Modèles vectoriels

- Modèle vectoriel de Salton
 - Ces scores sont calculés de façon itérative en parcourant la liste inversée de tous les termes de la requête.
 - La normalisation des scores pour des mesures de similarité comme la mesure cosinus est réalisée à la fin de cette boucle itérative.

3.1.2) Modèles vectoriels

- Modèle vectoriel de Salton
 - On remarque dans ce cas que le facteur normalisateur correspondant à la requête est estimé au cours de l'exécution de l'algorithme mais qu'il n'est pas nécessaire de procéder de même pour les facteurs normalisateurs des documents.

3.1.2) Modèles vectoriels

- Modèle vectoriel de Salton
 - En effet, ces facteurs dépendent directement des vecteurs de représentation des documents qui ne changent que lorsqu'il y a eu des ajouts ou des modifications dans le texte des documents.
 - La stratégie préconisée dans l'algorithme 3 consiste à mettre en mémoire le tableau contenant ces facteurs.

3.1.2) Modèles vectoriels

- Modèle vectoriel de Salton

- Une fois les scores des documents calculés, les L documents de plus haut score sont retournés comme résultat de recherche.
- Dans le cas où on considère la représentation *tf-idf* normalisée pour les documents,

$$\forall d, d = \left(\frac{tf_{t_i d}}{l_d} \times idf_{t_i} \right)_{i \in \{1, \dots, V\}} \quad \text{où } l_d = \sum_{t \in V} tf_{t_i d}$$

- et la représentation fréquentielle pour les requêtes,

$$\forall d, q = (tf_{t_i d})_{i \in \{1, \dots, V\}},$$

- on peut montrer que la fonction de similarité à base du produit scalaire

$$\forall d, \forall d, s(q, d) = \sum_{t \in d \cap q} tf_{t_i q} \times \frac{tf_{t_i d}}{l_d} idf_t \quad (2)$$

- vérifie les conditions de la RI.

3.1.3) Modèles probabilistes

3.1.3) Modèles probabilistes

- 3.1.3 Modèles probabilistes
- Il existe une troisième famille de méthodes de recherche qui, étant données les représentations d'un document et d'une requête, essaie d'inférer la **probabilité de pertinence** du document sachant la requête ou plus généralement la probabilité qu'une requête soit associée à un document.
- Ces modèles ont été proposés au début des années 60 (Maron et Kuhns 1960) et leur but principal est d'apporter **une explication probabiliste** au problème de la recherche documentaire.
-

3.1.3) Modèles probabilistes

- 3.1.3 Modèles probabilistes
- Dans la suite, nous allons présenter les **approches probabilistes** les plus courantes, à savoir :
 - le modèle d'indépendance binaire,
 - BM25,
 - les modèles de langue ainsi que
 - les modèles informationnels.

3.1.3) Modèles probabilistes

- Principe d'ordonnement probabiliste
- Avant de présenter les approches probabilistes, nous allons d'abord exposer le principe d'ordonnement sur lequel la majorité de ces approches sont fondées.
- Ce principe,
 - connu sous le nom du principe d'ordonnement probabiliste (*Probability Ranking Principle*) (Robertson 1977a,b),
 - considère chaque paire constituée d'un document et d'une requête (d,q) comme la réalisation d'une expérience aléatoire qui consiste à tirer simultanément ces observations d'un ensemble prédéfini de documents et de requêtes.

3.1.3) Modèles probabilistes

- **Principe d'ordonnement probabiliste**
- À chaque tirage (d, q) on associe alors une variable aléatoire (v.a.) binaire $R_{d,q}$ qui est égale à 1 si le document d est pertinent par rapport à la requête q et 0, sinon.
- Le but est alors :
 - de modéliser l'expérience aléatoire avec un modèle probabiliste donné ;
 - d'estimer les paramètres du modèle sur une base de requêtes et de documents pertinents, non pertinents associés à chacune des requêtes de cette base ;
 - et, pour une nouvelle requête q_{nouv} , de trier les documents d en fonction de leur probabilité *a posteriori*, $P(R|d, q_{nouv})$.

3.1.3) Modèles probabilistes

- Principe d'ordonnancement probabiliste
- La fonction de décision d'ordonnancement dépend de la fonction de coût choisie pour estimer les paramètres du modèle de recherche.
- Nous allons présenter les fonctions de coût usuelles utilisées avec les modèles de recherche que nous étudierons dans la suite.

•

3.1.3) Modèles probabilistes

- **Modèle d'indépendance binaire**
- Le modèle d'indépendance binaire (MIB) est le modèle classique associé au principe d'ordonnancement probabiliste.
- Ce modèle est fondé sur quatre hypothèses :

3.1.3) Modèles probabilistes

- Modèle d'indépendance binaire

H1.

- La première stipule que les documents et les requêtes sont représentés sous forme de vecteurs binaires de même taille que le vocabulaire de la collection.
- Si un terme du vocabulaire est présent dans un document (ou une requête), la caractéristique associée à ce terme dans le vecteur représentatif du document (ou de la requête) est fixée à **1** et à **0** dans le cas contraire.

3.1.3) Modèles probabilistes

- Modèle d'indépendance binaire

H2.

- La deuxième correspond à l'hypothèse **sac de mots** présentée au chapitre 2 et qui stipule que les termes présents dans un document sont **mutuellement indépendants**.
- En classification documentaire cette hypothèse est plus connue sous le nom de l'hypothèse de **Naive Bayes** et elle est à la base d'une famille de modèles de classification portant le même nom.;

3.1.3) Modèles probabilistes

- Modèle d'indépendance binaire

H3.

- La troisième hypothèse concerne la **forme de la fonction de coût d'ordonnement** qui comptabilise le nombre moyen de documents pertinents (respectivement non pertinents) qui sont retournés comme non pertinents (respectivement pertinents).
- Nous y revenons ci-dessous ;

3.1.3) Modèles probabilistes

- Modèle d'indépendance binaire

H4.

- La dernière hypothèse spécifie que tous les termes non présents dans la requête sont **uniformément répartis** dans les documents pertinents et non pertinents par rapport à cette dernière.

3.1.3) Modèles probabilistes

- **Modèle d'indépendance binaire**
- Sous l'**hypothèse H3**, la règle de décision optimale amenant à la minimisation du coût d'ordonnancement est celle qui consiste à sélectionner les documents pertinents vérifiant l'inégalité suivante :
 - le document d est pertinent par rapport à la requête q si et seulement si :

$$P(R = 1 \mid d, q) > P(R = 0 \mid d, q)$$

3.1.3) Modèles probabilistes

- Modèle d'indépendance binaire
- Comme le but final de la recherche est d'ordonner les documents par rapport à leur pertinence avec la requête, la règle précédente est appliquée en examinant le rapport des probabilités *a posteriori*, qui, d'après le théorème de Bayes, s'écrit :

$$\Delta(d, q) = \frac{p(R=1|d, q)}{p(R=0|d, q)} = \frac{p(d|R=1, q)}{p(d|R=0, q)} \times \frac{p(R=1|q)}{p(R=0|q)}$$

(3)

3.1.3) Modèles probabilistes

- **Modèle d'indépendance binaire**
- Plus ce rapport est élevé pour un document d , plus le document est considéré comme pertinent par rapport à la requête q .
- Autrement dit, ce rapport de probabilités joue le même rôle qu'une fonction de score utilisée ou apprise dans les modèles vectoriels.

-

3.1.3) Modèles probabilistes

- **OKAPI BM25**
- Le modèle OKAPI BM25 (dénommé plus simplement BM25 7) (Robertson et Walker 1994 ; Zaragoza *et al.* 2004) est devenu une référence dans le développement des systèmes de recherche.
- L'idée de départ de ce modèle est qu'un bon descripteur de document est un terme assez fréquent de ce document mais qui est relativement rare dans la collection contenant ce document.
- Ce terme représenterait alors le document relativement bien, tout en le distinguant des autres documents de la collection.

3.1.3) Modèles probabilistes

- **OKAPI BM25**
- Cette idée est fondée sur le constat que beaucoup de termes apparaissent avec une fréquence assez basse dans beaucoup de documents d'une collection, alors qu'ils apparaissent avec une fréquence élevée dans un groupe distinct de documents, généralement appelé groupe *élite*.
- Ce constat a motivé la modélisation du groupe élite avec la loi de Poisson de paramètre λ_E , ainsi que la modélisation du groupe non élite avec la loi de Poisson de paramètre λ_{NE} et respectant la condition $\lambda_{NE} < \lambda_E$.
- La probabilité d'apparition d'un terme $ti \in V$ apparaissant $tfti$, d fois dans un document d peut être ainsi exprimée par une loi de mélange de paramètre α_i ou β_i ^B (connue sous le nom de la loi 2-Poisson) suivant que ce document a été jugé pertinent ou non par rapport à une requête q (dans le cas général, nous avons : $\forall i, \alpha_i > \beta_i$) :

3.1.3) Modèles probabilistes

- OKAPI BM25**

$$P(w_{id} = tf_{t_id} | R = 1, q) = \alpha_i \frac{\lambda_E^{tf_{t_id}} e^{-\lambda_E}}{tf_{t_id}!} + (1 - \alpha_i) \frac{\lambda_{NE}^{tf_{t_id}} e^{-\lambda_{NE}}}{tf_{t_id}!} \quad (7)$$

et

$$P(w_{id} = tf_{t_id} | R = 0, q) = \beta_i \frac{\lambda_E^{tf_{t_id}} e^{-\lambda_E}}{tf_{t_id}!} + (1 - \beta_i) \frac{\lambda_{NE}^{tf_{t_id}} e^{-\lambda_{NE}}}{tf_{t_id}!} \quad (8)$$

3.1.3) Modèles probabilistes

- Modèles de langue
- Dans les modèles probabilistes précédents, la notion de pertinence apparaît explicitement dans l'expression de la fonction de score.
- En effet, nous avons vu que dans ce cas le but est d'estimer la probabilité de pertinence pour un document et une requête utilisateur donnés : $P(R = 1 \mid d, q)$.
-

3.1.3) Modèles probabilistes

- Modèles de langue
- Une approche alternative au problème de recherche consiste à considérer qu'un document et une **requête** sont **proches si et seulement si, il est facile de *générer* la requête à partir du document.**
- Il n'y a dans ce cas aucune référence explicite à la notion de pertinence.
- Les modèles construits suivant cette idée sont connus sous le nom des ***modèles de langue*** et ils supposent qu'une requête ***q*** est **générée par un modèle probabiliste associé à un document *d***
-

3.1.3) Modèles probabilistes

- Modèles de langue

Méthode	$P(t \mid d)$
Jelinek-Mercer	$(1 - \lambda)P_{mv}(t \mid d) + \lambda P_{mv}(t \mid C)$
Decompte absolu	$\frac{\max(\text{tf}_{t,d} - \alpha, 0)}{\sum_{t \in V} \text{tf}_{t,d}} + \beta P_{mv}(t \mid C)$
MAP	$\frac{\text{tf}_{t,d} + \gamma P_{mv}(t \mid C)}{\sum_{t \in V} \text{tf}_{t,d} + \gamma}$

Tab. 3.1 – Les trois méthodes d'estimation de la probabilité d'apparition d'un terme dans un document les plus répandues dans les modèles de langue.

3.1.3) Modèles probabilistes

- La méthode Jelinek-Mercer
- Cette méthode
 - était proposée en premier pour des modèles de langue en reconnaissance de la parole (Jelinek 1997) et
 - elle consiste
 - à estimer la probabilité conditionnelle de présence d'un terme t dans un document d , $P(t | d)$,
 - à l'aide d'une interpolation linéaire entre les probabilités conditionnelles du terme dans la collection et dans le document, obtenues toutes les deux avec la méthode du maximum de vraisemblance, et notées respectivement $Pmv(t | C)$ et $Pmv(t | d)$.

3.1.3) Modèles probabilistes

- La méthode du **m**aximum **a** **p**osteriori (MAP) avec **a** priori de Dirichlet
- Cette méthode suppose que les termes présents dans un document sont issus d'un tirage aléatoire avec remise suivant une distribution multinomiale.
- Dans ce cas, les termes t du vocabulaire sont échantillonnés avec des probabilités $P(t \mid d)$ qui sont estimées suivant la méthode du maximum *a posteriori* avec un *a priori* de Dirichlet sur les paramètres du modèle.....

3.1.3) Modèles probabilistes

- **Modèles informationnels**
- La plupart des modèles de RI n'utilisent pas simplement le nombre d'occurrences d'un terme dans un document mais plutôt une normalisation de ce nombre.
- Par exemple, les modèles de langue utilisent, comme nous venons de le voir, la fréquence relative des termes dans un document et dans la collection.....

<http://www.iro.umontreal.ca/~nie/IFT6255/>